**PROJECT REPORT OF INDUSTRY ORIENTED HANDS-ON EXPERIENCE (IOHE)**

**ON**

# Stackraze: A Dev Community Platform

**submitted in partial fulfilment of the requirements for the award of the degree of**

**BACHELOR OF ENGINEERING**

**In**

**COMPUTER SCIENCE AND ENGINEERING**


**Submitted by:**

**Shreya Yadav, 2210990841**

**Palak Bisht, 2210990631**

**Tanisha Singla, 2210990893**

**Supervised By:**

**Dr. Monika Aggarwal**

**Assistant Professor**

**Chitkara University, Punjab**

**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING**

**CHITKARA UNIVERSITY INSTITUTE OF ENGINEERING AND TECHNOLOGY**
**CHITKARA UNIVERSITY, PUNJAB, INDIA**

# CONTENTS


| S. No. | Title | Page No. |
|---|---|---|
| | Acknowledgement | iv |
| | List of Figures and Tables | v |
| | Abstract | vi |
| 1 | Introduction | 1 |
| 2 | Methodology | 4 |
| 3 | Tools and Technologies | 5 |
| 4 | Implementation | 6 |
| 5 | Expected Outcome | 8 |
| 6 | Conclusion and Future Scope | 18 |
| | References | 19 |
| | Appendices | 20 |

# DECLARATION

I hereby certify that the work which is being presented in the project report entitled **"Stackraze – A Dev Community Platform"** in partial fulfilment of requirement for the award of the degree of Bachelor of Engineering (Computer Science and Engineering) submitted in the Department of Computer Science and Engineering at Chitkara University Institute of Engineering and Technology, Chitkara University, Punjab, India, is an authentic record of our own work carried out under the supervision of **Dr. Monika Aggarwal, Assistant Professor**. The matter presented in this project report has not been submitted to any other university/institute for the award of any degree.

Place: Rajpura

Date: 29/04/2026

**Shreya Yadav**

University Roll No: 2210990841

**Palak Bisht**

University Roll No: 2210990631

**Tanisha Singla**

University Roll No: 2210990893

**Dr. Monika Aggarwal**

Assistant Professor

Department of Computer Science and Engineering

Chitkara University Institute of Engineering and Technology,

Chitkara University, Punjab, India

# ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who supported and guided me throughout the development of the Stackraze project.

First and foremost, I am deeply thankful to my mentor, Ms. Monika Agarwal, Assistant Professor at Chitkara University, for her continuous guidance, valuable insights, and encouragement. Her constructive feedback and technical expertise played a crucial role in shaping the direction and successful completion of this project.

I extend my heartfelt thanks to the faculty members of the Computer Science and Engineering department for providing the necessary academic support and resources required during the course of this work.

I would also like to acknowledge Chitkara University for offering a platform to work on a project like Stackraze, which focuses on practical implementation and real-world problem-solving. Such opportunities greatly enhance technical knowledge and professional skills.

I am grateful to my teammates and peers for their collaboration, idea sharing, and consistent support throughout the development process. Their contributions helped in overcoming challenges and improving the overall quality of the project.

Finally, I would like to thank my family for their constant encouragement, patience, and unwavering support, which motivated me to successfully complete this project.

The successful completion of this project would not have been possible without the guidance and support of all the above-mentioned individuals.

# LIST OF FIGURES AND TABLES

## FIGURES

Fig. 1 System Architecture ..... 2

Fig. 2 Sign Up Page ..... 8
Fig. 3 Sign In Page ..... 8
Fig. 4 Landing Page ..... 9
Fig. 5 About Section ..... 10
Fig. 6 FAQs Section ..... 11
Fig. 7 Landing Page (Dark Mode) ..... 12
Fig. 8 Home Page (Light and Dark Mode) ..... 13
Fig. 9 Newsletter Page ..... 14
Fig. 10 Community Page ..... 14
Fig. 11 Connect Page ..... 15
Fig. 12 Contact Page ..... 15
Fig. 13 Profile Page ..... 16
Fig. 14 Mobile View of Website ..... 17

Fig. A1 Core System Workflow of Stackraze Platform ..... 23
Fig. A2 Real-Time Chat Communication Flow using Socket.io ..... 24
Fig. A3 Project Folder Structure ..... 25

## TABLES

Table I Frontend Technology Stack ..... 5
Table II Backend Technology Stack ..... 5

Table A1 Authentication API Endpoints ..... 20
Table A2 Post and Query APIs ..... 20
Table A3 Saved Posts APIs ..... 21
Table A4 Geo / Connect APIs ..... 21
Table A5 Community APIs ..... 21

# ABSTRACT

Stackraze, a full-stack web application developed using the MERN (MongoDB, Express.js, React.js, Node.js) stack, aimed at improving the management and organization of structured data and tasks through a centralized platform. The system provides a scalable and user-friendly interface that enables efficient data handling, visualization, and real-time interaction. The frontend is implemented using React.js to deliver a responsive and dynamic user experience, while the backend is built with Node.js and Express.js to manage application logic and RESTful API communication. MongoDB is utilized as the database for flexible and efficient storage of user and application data. The application incorporates secure user authentication and authorization using JSON Web Tokens (JWT) and bcrypt-based password encryption. Key features include a dynamic dashboard, CRUD operations, and protected routes for secure access control. The system follows a modular architecture to ensure maintainability and scalability. Experimental results demonstrate improved efficiency in data organization and user interaction. The proposed solution highlights the effectiveness of modern web technologies in building robust and scalable applications for real-world use cases.

# 1. Introduction

## 1.1 Problem Background

Modern software developers frequently encounter technical challenges such as debugging errors, dependency conflicts, and system-level issues that disrupt productivity and delay development cycles. Existing support platforms, including online forums, documentation sites, and AI-based tools, often provide fragmented, delayed, or context-insensitive solutions.

Developers are required to switch between multiple platforms to find relevant answers, leading to inefficiency, redundancy, and lack of accountability in problem resolution. Additionally, many complex issues require contextual understanding and human expertise, which fully automated systems are unable to consistently deliver.

Therefore, there is a need for a unified and structured platform that combines intelligent assistance with collaborative human support, enabling faster, more reliable, and context-aware resolution of technical problems.

## 1.2 Objective of the Project

The primary objectives of *"Stackraze – Hybrid Developer Support Platform"* are:

- To develop a centralized platform for developers to seek and provide solutions efficiently
- To reduce issue resolution time through integration of intelligent assistance and community expertise
- To enable context-aware problem solving by allowing users to share logs, errors, and environment details
- To facilitate real-time collaboration through interactive features such as chat systems
- To build a structured knowledge repository for future reference and reuse
- To ensure secure and reliable user authentication and data handling
- To promote knowledge sharing and collaboration among developers
- To design a scalable and modular system architecture

## 1.3 Proposed Solution / System Overview

Stackraze is a web-based developer support platform that integrates intelligent assistance with structured community collaboration. The system follows a modular full-stack architecture consisting of a Reactbased frontend, a Node.js and Express backend, and a MongoDB database, with Socket.io enabling real- time communication.

The platform provides secure user authentication using JWT and OTP-based verification mechanisms. Users can submit technical queries with contextual details such as logs and descriptions, which are organized and processed efficiently. The system supports both automated guidance and community-based problem resolution, ensuring accurate and timely solutions.

A real-time chat system enables instant communication among users, facilitating collaborative problemsolving. All resolved queries are stored to build a continuously evolving knowledge repository.

## 1.4 System Architecture

The system follows a three-tier architecture consisting of frontend, backend, and database layers. The React.js frontend handles user interaction, while the Node.js and Express backend processes application logic and API requests. MongoDB is used for data storage, and Socket.io enables real-time communication.

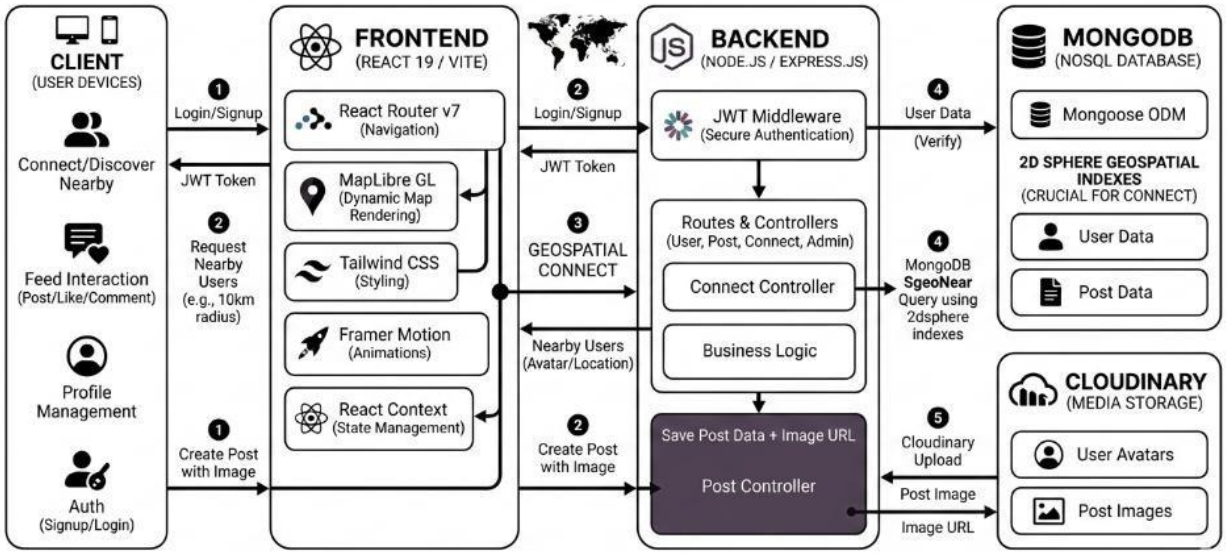


Fig 1: Showing System Architecture

## 1.5 System Modules

The Stackraze platform is divided into multiple functional modules to ensure scalability and maintainability:

- Authentication Module – Handles signup, login, JWT authentication, and OTP verification
- User Management Module – Manages user profiles and account data
- Query Management Module – Handles posting and categorization of queries
- Community Interaction Module – Enables user engagement and collaboration
- Real-Time Chat Module – Supports instant messaging using Socket.io

- Connect Module – Provides location-based developer interaction
- Notification Module – Manages system alerts and updates
- Database Module – Handles data storage using MongoDB
- Backend API Module – Processes business logic using Node.js and Express
- Frontend Module – Provides user interface using React

## 1.6 Key Features

The Stackraze platform includes the following key features:

- Structured query submission with contextual details
- Context-aware problem solving
- Hybrid support system combining automation and community expertise
- Real-time communication using Socket.io
- Persistent knowledge repository
- Secure authentication using JWT and OTP verification
- Location-based developer connectivity

## 2. Methodology

The development of *Stackraze* follows a structured and modular approach using the MERN stack (MongoDB, Express.js, React.js, Node.js). The system is designed to ensure scalability, security, and efficient performance, with cloud deployment on Vercel for high availability and seamless updates.

### 2.1 System Workflow

- Users access the platform through a web browser
- The React.js frontend handles user interaction
- Requests are sent to backend APIs
- The Node.js and Express.js backend processes logic
- Authentication is managed using JWT
- Data is stored and retrieved from MongoDB
- Responses are displayed dynamically on the frontend

### 2.2 Development Approach

- **Requirement Analysis:** Identified key features such as authentication, real-time chat, and query management
- **System Design:** Implemented a three-tier architecture for scalability and modularity
- **Frontend Development:** Built a responsive UI using React.js with reusable components
- **Backend Development:** Developed RESTful APIs with secure authentication
- **Database Integration:** Used MongoDB with Mongoose for efficient data handling
- **Deployment:** Deployed on Vercel with GitHub integration for continuous updates
- **Testing:** Conducted API and UI testing, followed by performance optimization

## 3. Tools and Technologies

The system employs a full-stack web architecture with modular backend services, secure authentication mechanisms, and scalable database integration.

**TABLE I: FRONTEND TECH STACK**

| Technology | Purpose |
|---|---|
| React 19 | UI framework for building interactive components |
| Vite 7 | Fast build tool and development server |
| React Router DOM v7 | Handles client-side routing/navigation |
| Tailwind CSS v4 | Utility-first CSS framework for styling |
| Framer Motion | Animations and smooth UI transitions |
| Lucide React | Icon library for modern UI icons |
| MapLibre GL | Interactive map integration for the Connect feature |
| Axios | HTTP client for API requests |

**TABLE II: BACKEND TECH STACK**

| Technology | Purpose |
|---|---|
| Node.js + Express 5 | Backend framework for building REST APIs |
| MongoDB + Mongoose | NoSQL database with ODM for schema management |
| JSON Web Token (JWT) | Stateless authentication and secure user sessions |
| Socket.io | Real-time communication using WebSockets |
| .env | Environment variable configuration management |
| Nodemon | Development tool for automatic server restarts |
| EJS | Server-side rendering for error pages (e.g., 404) |
| CORS | Enables cross-origin resource sharing between frontend and backend |

# 4. Implementation

## 4.1 Development Environment

The system was developed using modern web technologies. The frontend was implemented using React.js, while the backend was developed using Node.js and Express.js. MongoDB was used as the database, and development was carried out using Visual Studio Code. Version control was managed using Git and GitHub.

## 4.2 Frontend Implementation

The frontend of Stackraze was developed using React.js with a component-based architecture. Reusable components were created for authentication, dashboard, chat interface, and user profile. State management was handled efficiently to ensure smooth user interaction and dynamic rendering of data. API integration was implemented using fetch/axios to communicate with backend services.

## 4.3 Backend Implementation

The backend was implemented using Node.js and Express.js to handle routing, business logic, and request processing. RESTful APIs were developed for user authentication, post management, and community interactions. Middleware was used for request validation and security.

## 4.4 Database Implementation

MongoDB was used as the database to store user data, posts, and messages. Mongoose was used for schema design and data validation. Collections such as users, posts, and messages were structured to ensure efficient storage and retrieval.

## 4.5 Authentication and Security

Secure authentication was implemented using JSON Web Tokens (JWT). Users are authenticated during login, and a token is generated to maintain session state. OTP-based verification was also integrated to enhance security. Passwords are securely stored using encryption techniques.

## 4.6 Real-Time Communication Implementation

Real-time communication was implemented using Socket.io, enabling instant message exchange between users. The system supports live chat, message broadcasting, and synchronization across multiple clients, ensuring smooth and interactive communication.

## 4.7 API Integration

The frontend communicates with the backend through REST APIs. These APIs handle operations such as login, signup, post creation, and data retrieval. Proper error handling and response management were implemented to ensure reliability.

## 4.8 Deployment Implementation

The application was deployed using Vercel for frontend hosting, ensuring fast and reliable access. The backend services were configured to handle API requests efficiently, and integration with GitHub enabled continuous deployment and version control.

# 5. Expected Outcome

Stackraze is expected to enhance developer productivity by minimizing redundant troubleshooting and improving the accuracy and traceability of solutions. The platform promotes collaborative learning, strengthens technical confidence, and establishes an auditable knowledge ecosystem for continuous improvement across developer communities.

It will also reduce dependency on scattered resources and provide a more structured approach to problem-solving. Over time, the platform can evolve into a reliable hub for developer support and knowledge sharing.

Here are the Screenshots attached.:

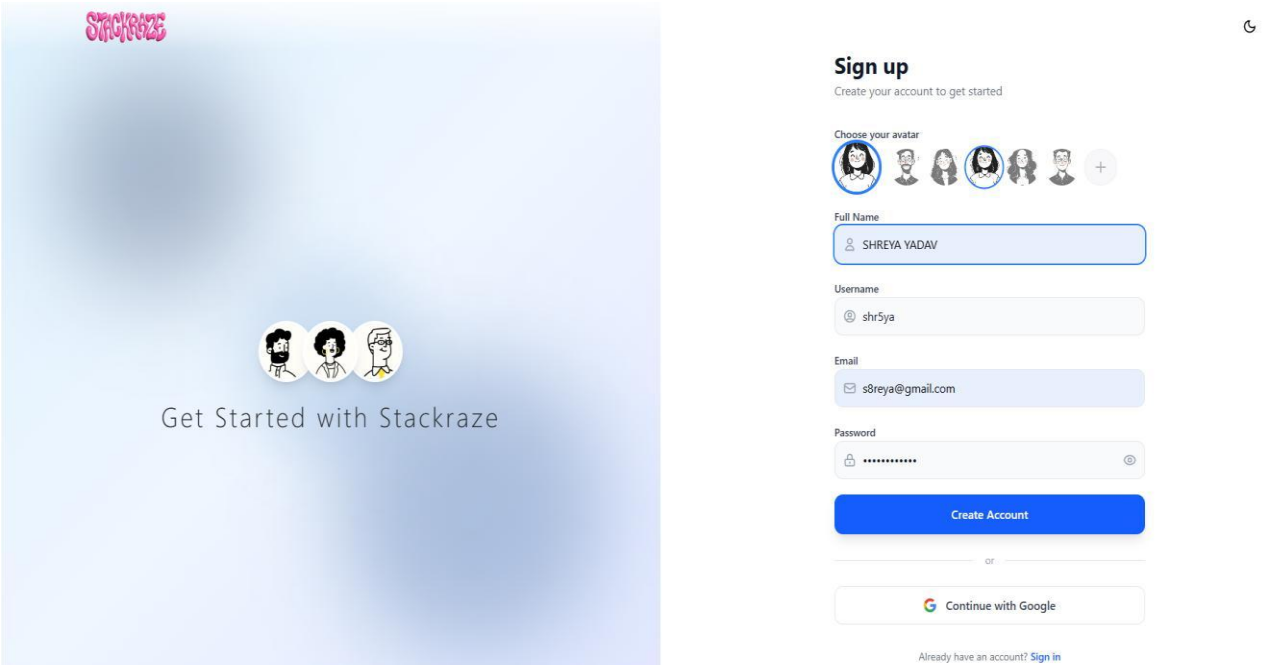


Fig 2: Sign up

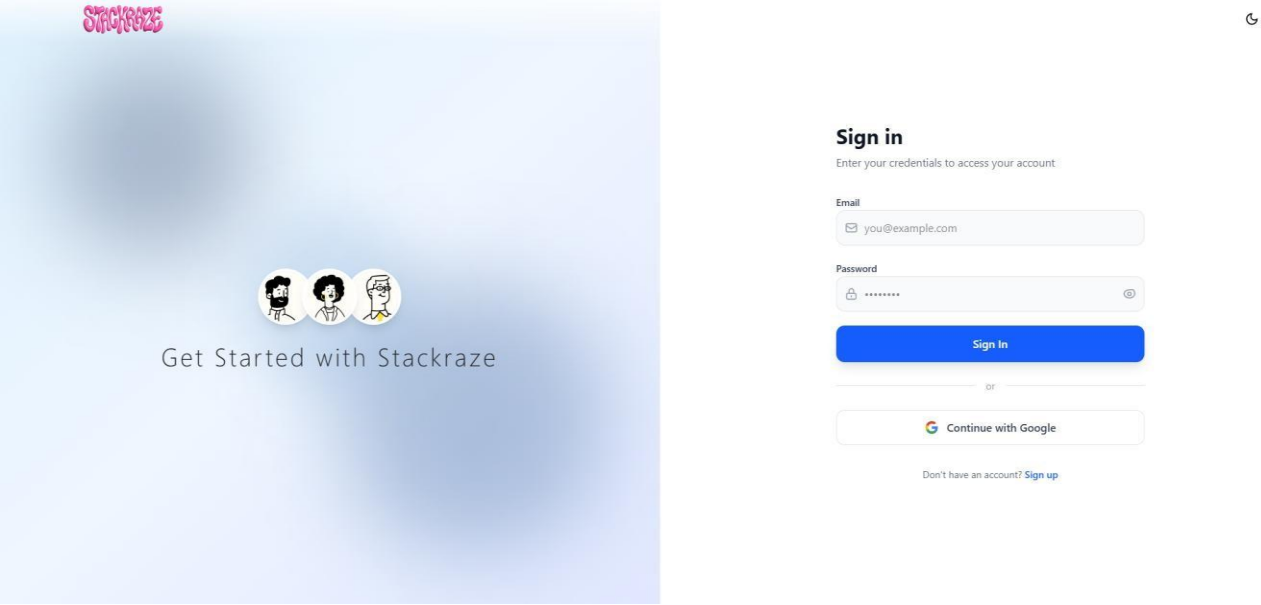


Fig 3: Sign in

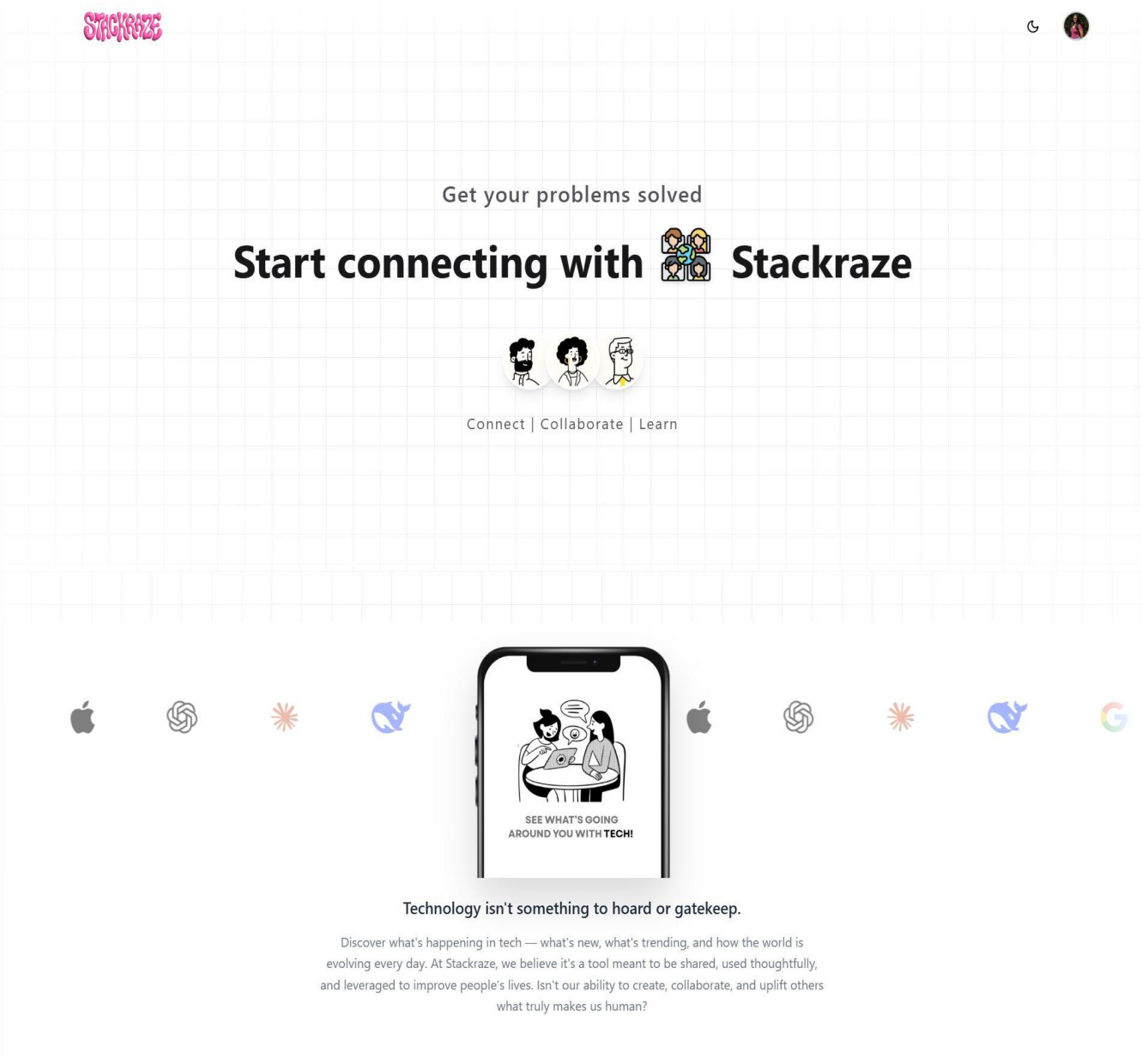


Fig 4: Landing

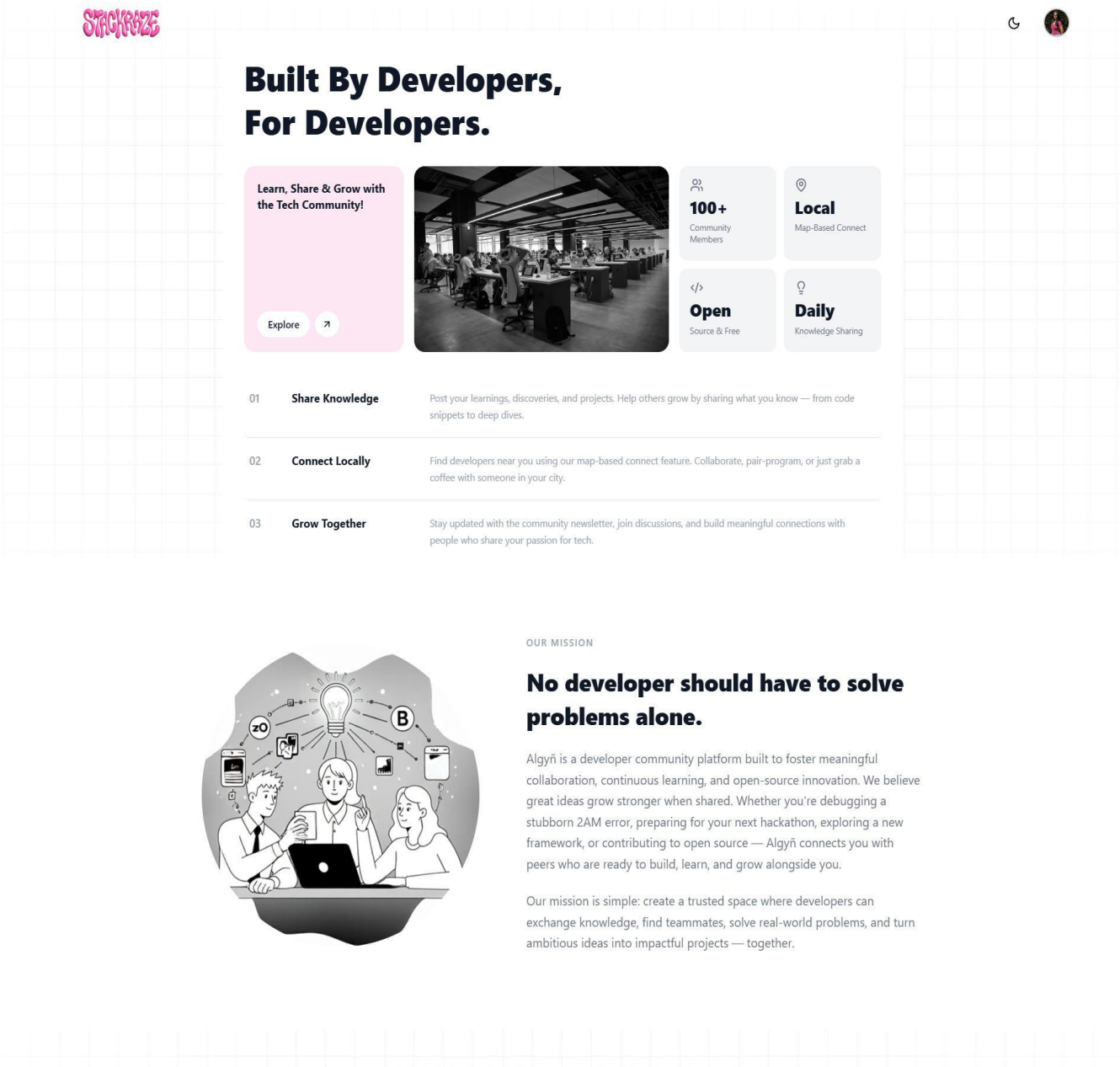


Fig 5: About Section

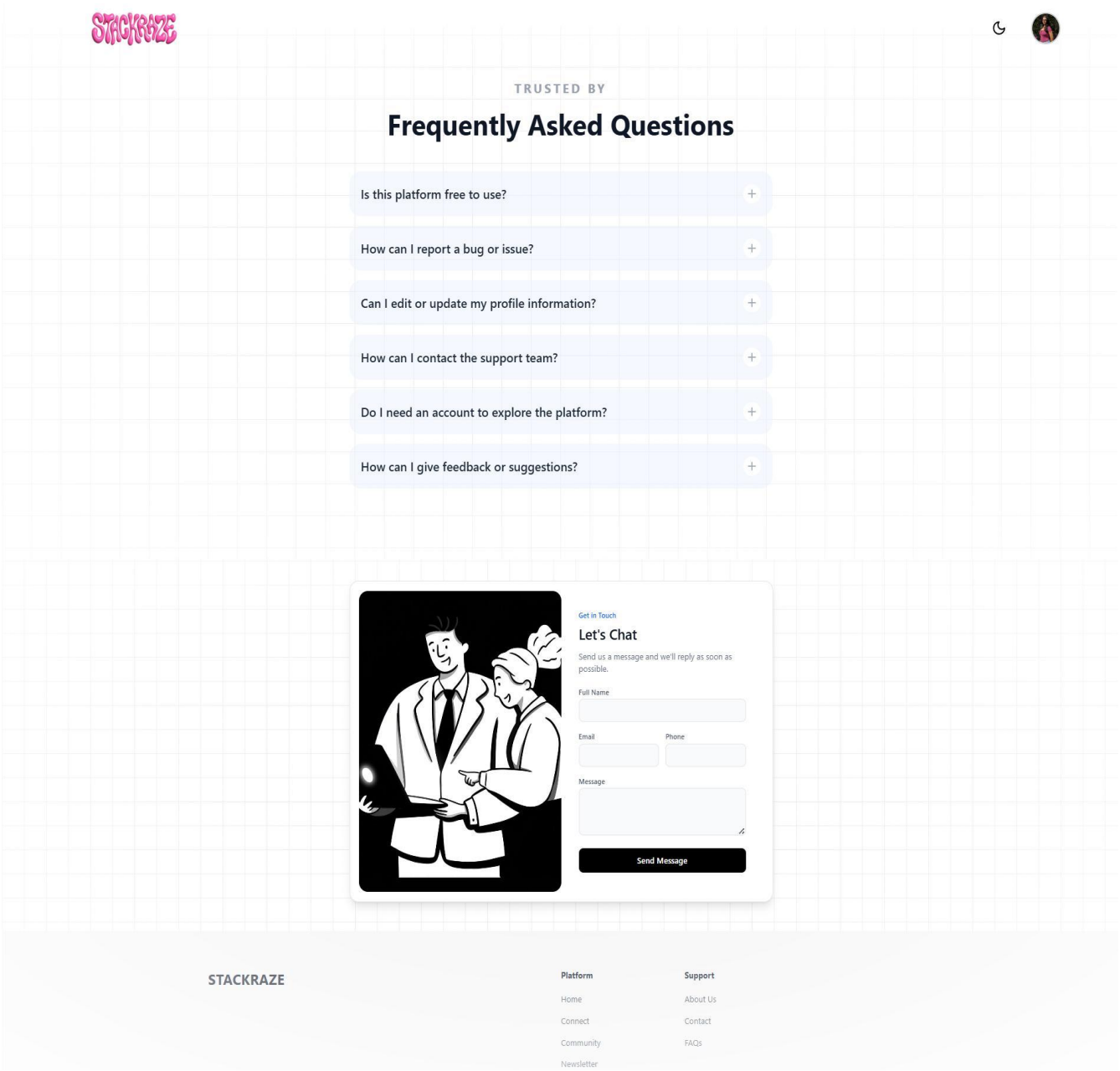


Fig 6: FAQs Section

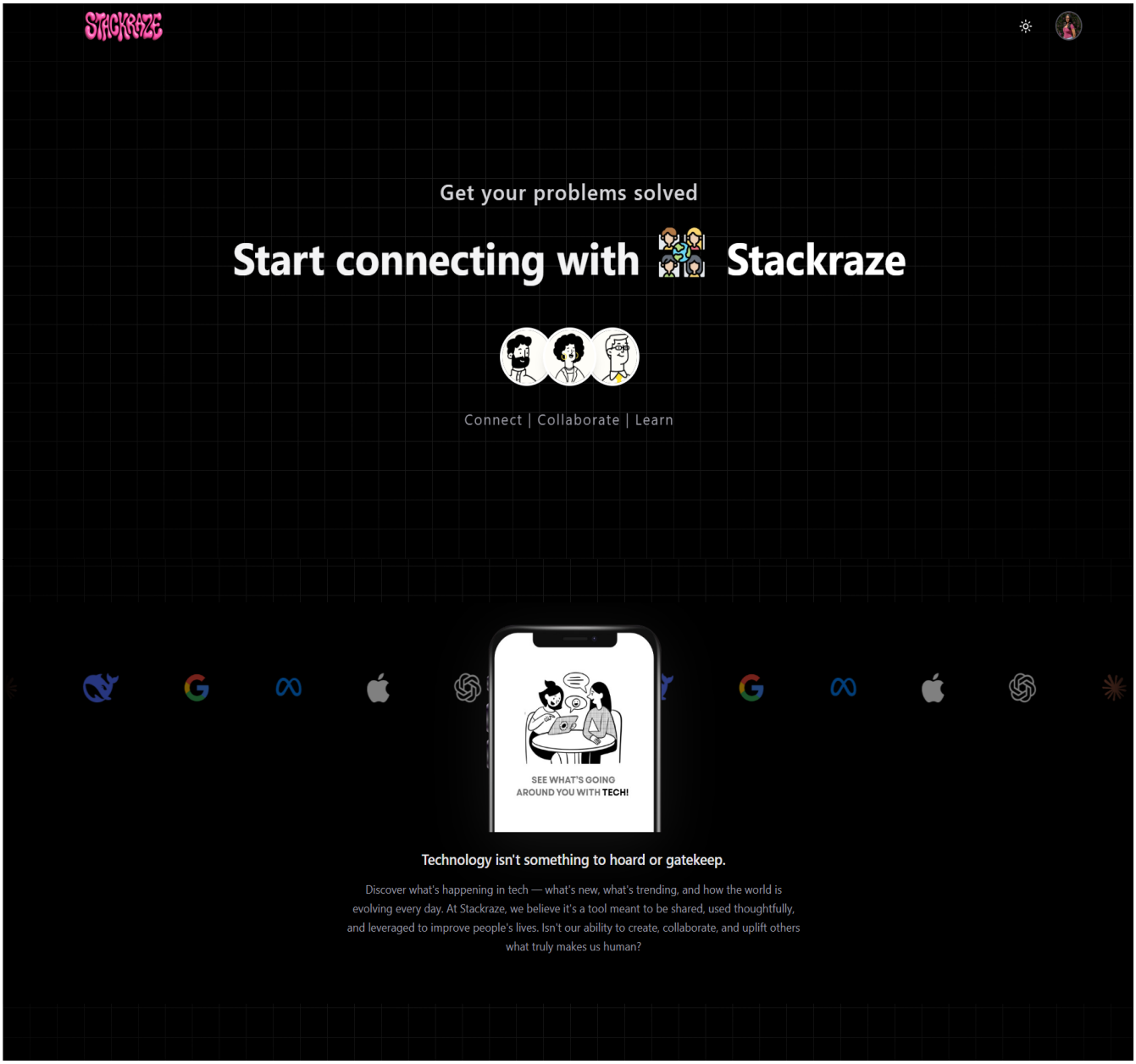


Fig 7: Landing Page (Dark Mode)

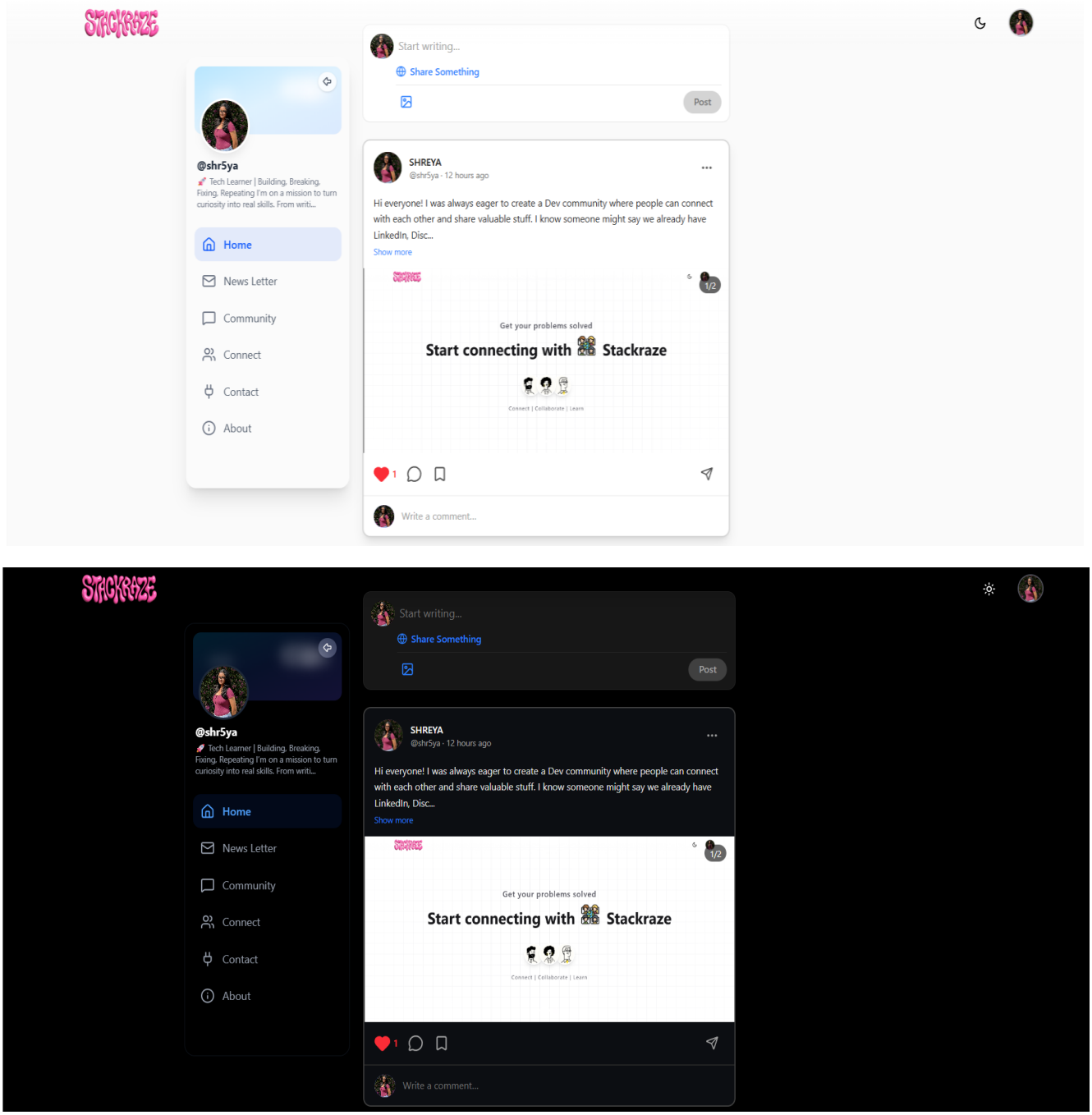


Fig 8: Home Page (Light and Dark mode)

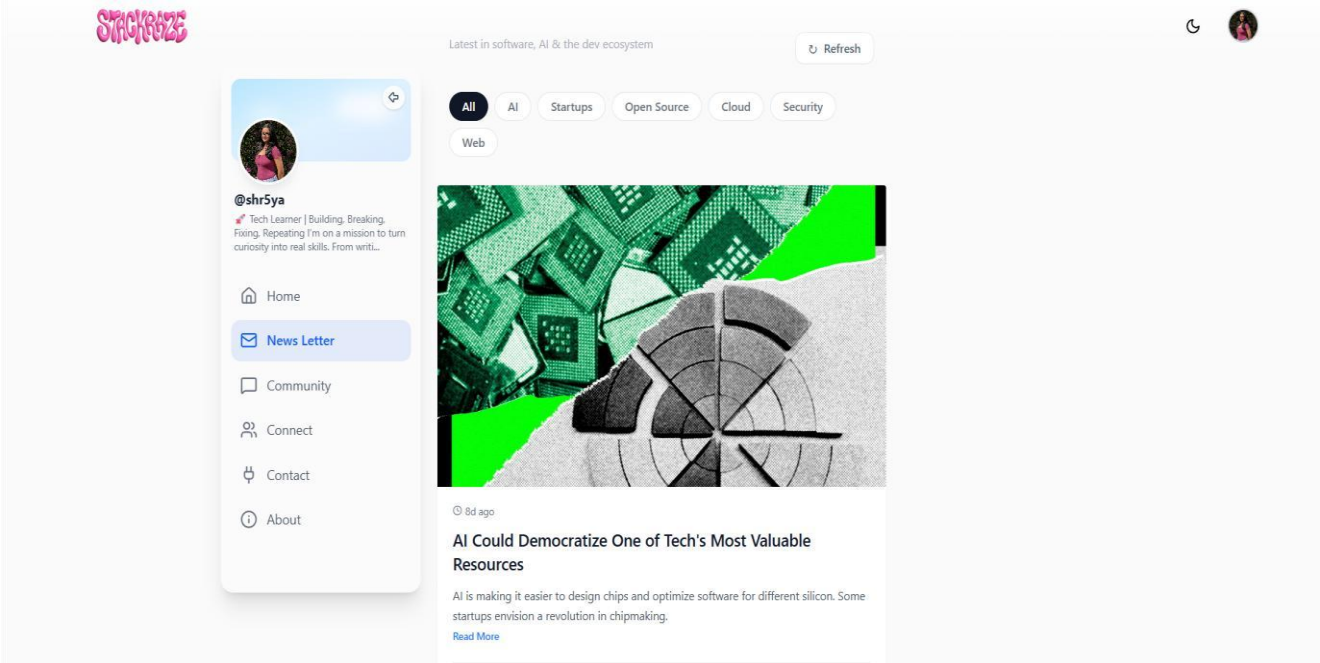


Fig 9: Newsletter Page

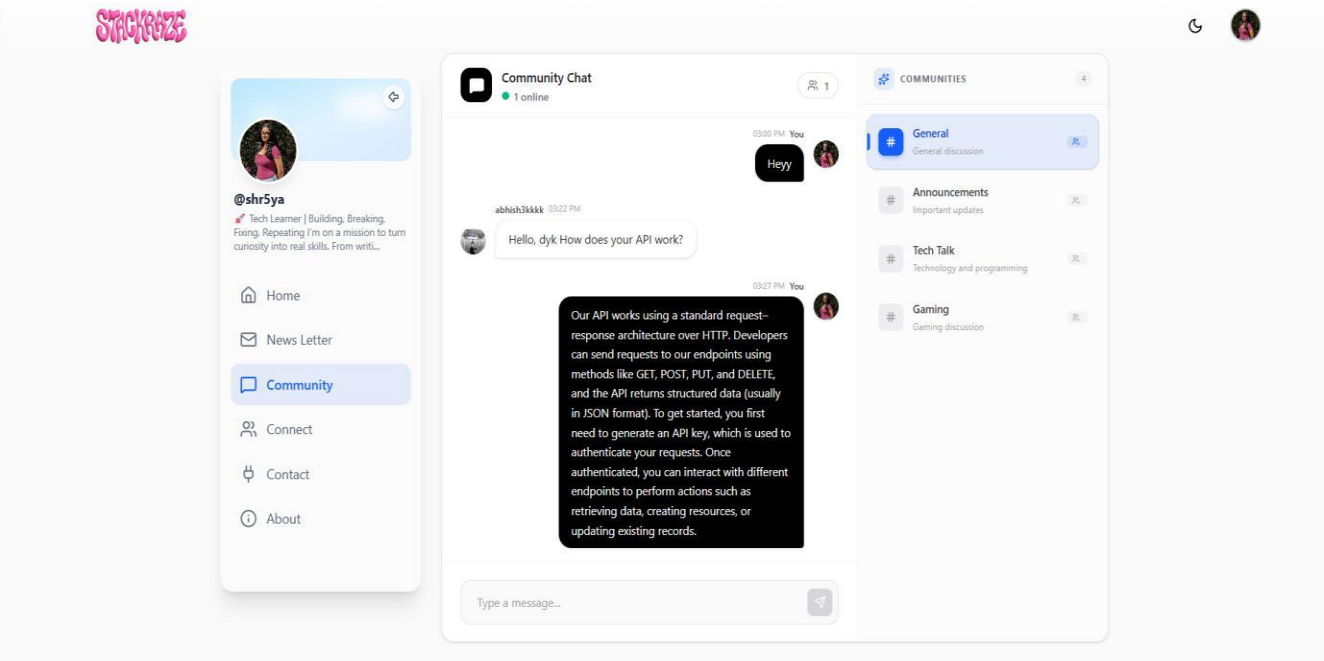


Fig 10: Community Page

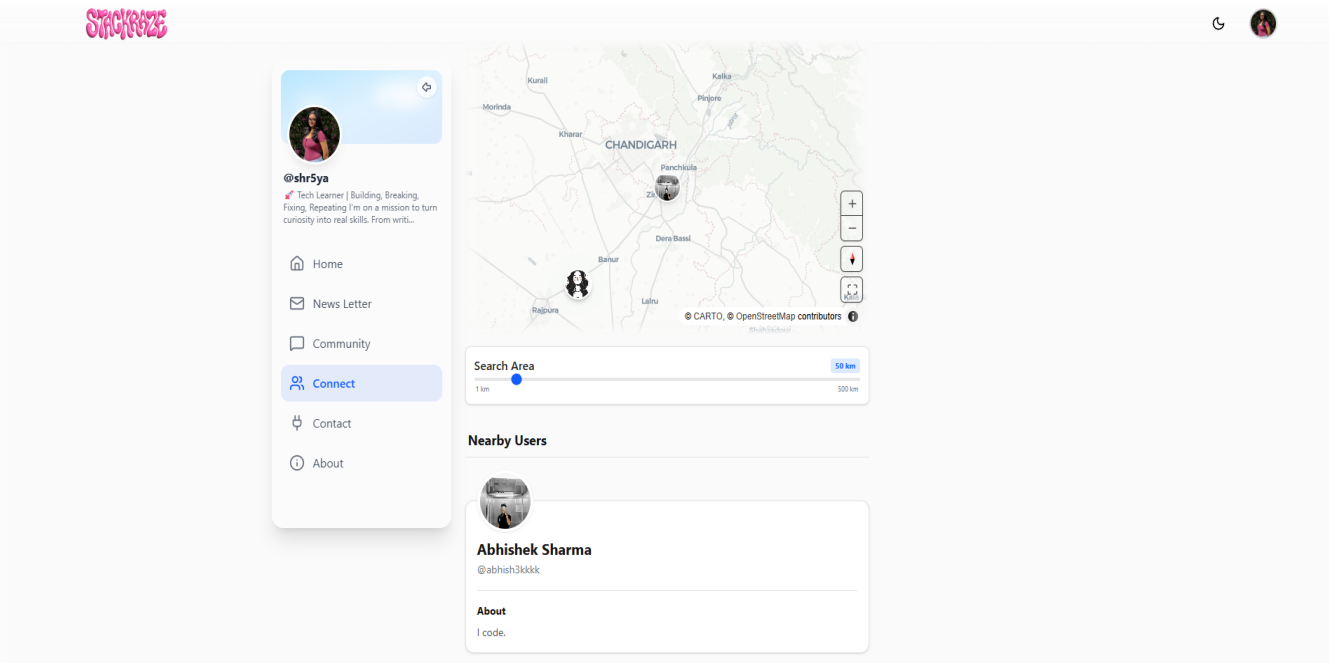


Fig 11: Connect Page

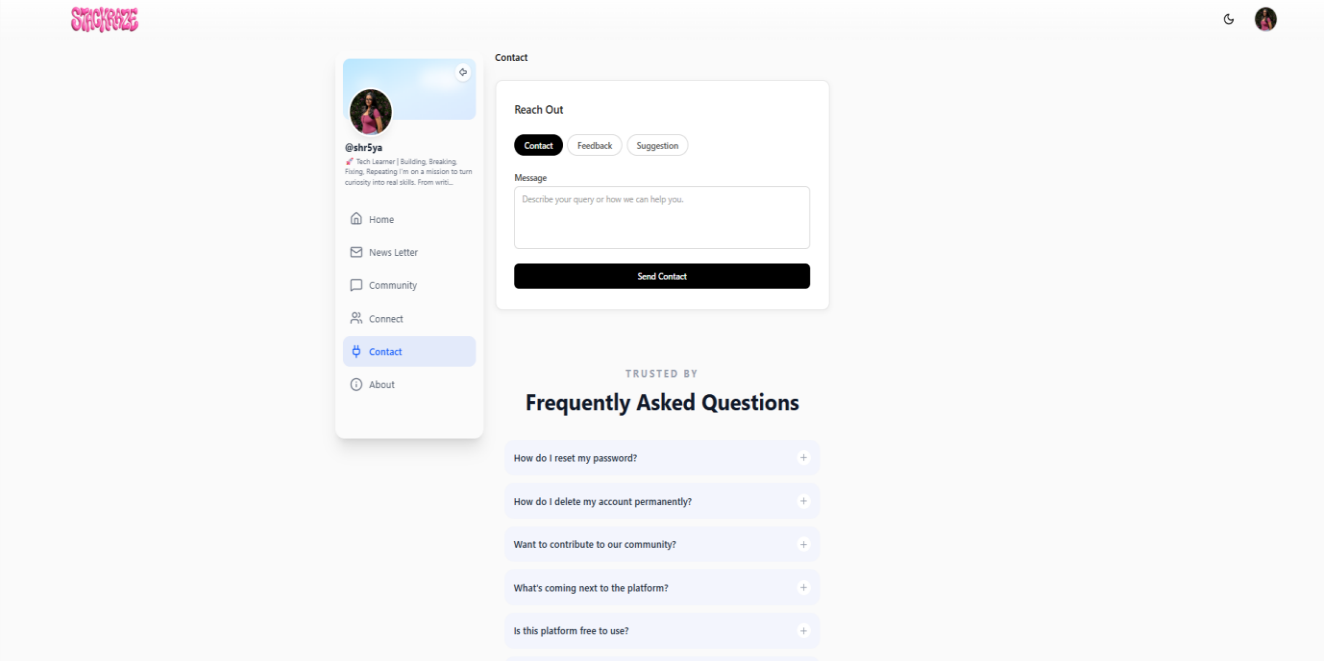


Fig 12: Contact Page

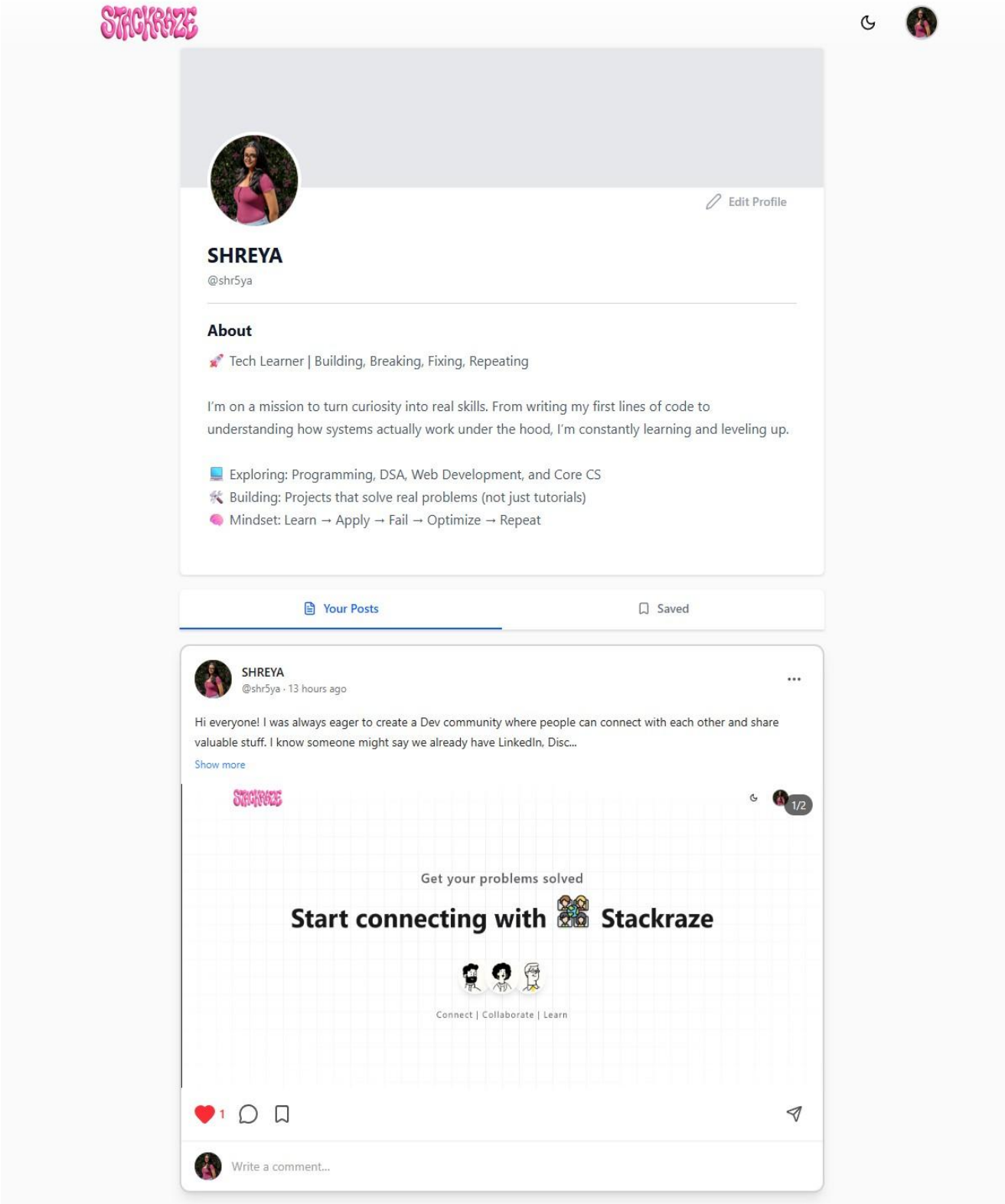


Fig 13: Profile Page

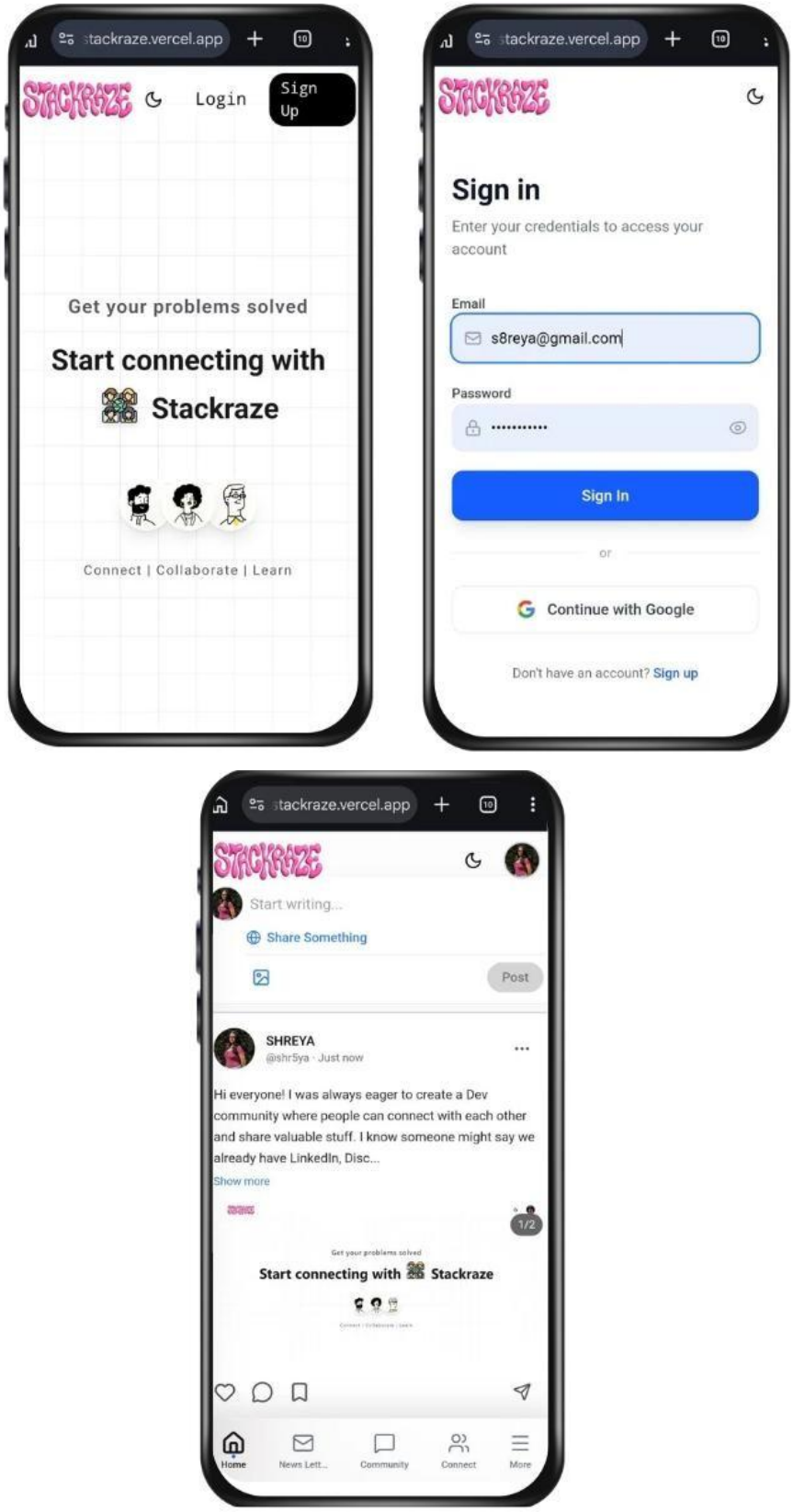


Fig 14: Mobile View of Website

# 5. Conclusion and Future Scope

## 5.1 Conclusion

Stackraze is a hybrid developer-support platform designed to enhance the efficiency and reliability of problem-solving within the developer community. By integrating structured query handling, real-time communication, and intelligent system design, the platform addresses key limitations of existing solutions, such as delayed responses, lack of accuracy, and unstructured knowledge sharing.

The system successfully combines modern web technologies with scalable architecture to provide a seamless user experience. Features such as authentication, community interaction, real-time chat, and location-based connectivity contribute to a comprehensive and interactive ecosystem for developers.

Overall, Stackraze demonstrates a practical and innovative approach to improving collaboration, knowledge reuse, and support accessibility, making it a valuable contribution in the domain of developer assistance.

## 5.2 Future Scope

The current implementation of Stackraze provides a strong foundation, with several opportunities for further enhancement:

- Integration of advanced AI-based debugging and recommendation systems to provide automated, context-aware solutions
- Development of a mobile application to increase accessibility and user engagement
- Expansion of the expert network to improve response quality and reduce resolution time
- Implementation of a reputation and reward system to encourage active community participation
- Enhancement of security features, including advanced authentication and data protection mechanisms
- Optimization of performance and scalability to support a larger user base
- Addition of personalized content and recommendation features based on user activity and interests

With these improvements, Stackraze can evolve into a more intelligent, scalable, and widely adopted platform for developer collaboration and support.

# References


1. React Documentation, "React – A JavaScript library for building user interfaces," Meta Platforms, Inc. [Online]. Available: https://react.dev
2. Node.js Foundation, "Node.js Documentation," OpenJS Foundation. [Online]. Available: https://nodejs.org
3. Express.js Documentation, "Express – Fast, unopinionated web framework for Node.js." [Online]. Available: https://expressjs.com
4. MongoDB Inc., "MongoDB Documentation," MongoDB Inc. [Online]. Available: https://www.mongodb.com
5. Socket.IO Documentation, "Socket.IO – Real-time bidirectional communication." [Online]. Available: https://socket.io
6. JSON Web Token (JWT), "JSON Web Token (JWT) RFC 7519," IETF, 2015.
7. Vercel Documentation, "Vercel Platform Documentation." [Online]. Available: https://vercel.com
8. GitHub, "GitHub Documentation." [Online]. Available: https://docs.github.com
9. Axios Documentation, "Axios – Promise-based HTTP client." [Online]. Available: https://axioshttp.com
10. Tailwind CSS Documentation, "Tailwind CSS." [Online]. Available: https://tailwindcss.com

# Appendices

## APPENDIX A: GITHUB REPOSITORY DETAILS

Repository URL:

https://github.com/shr5ya/StackRaze_2210990841_2210990631_2210990893 Repository

Structure Overview:

- /Frontend → React.js UI implementation
- /Backend → Node.js + Express API
- /models → MongoDB schemas
- /controllers → Business logic layer
- /routes → API route definitions
- /sockets.js → Real-time communication setup

## APPENDIX B: COMPLETE API ENDPOINT STRUCTURE

Table A1 Authentication API Endpoints

| Method | Endpoint | Description |
|---|---|---|
| POST | /user/signup | Register user |
| POST | /user/login | Login + JWT |
| POST | /user/verify-otp | OTP verification |
| POST | /user/resend-otp | Resend OTP |
| GET | /user/userData | Get logged-in user |
| PATCH | /user/updateData | Update profile |

Table A2 Post and Query APIs

| Method | Endpoint | Description |
|---|---|---|
| POST | /user/post/create | Create post |
| GET | /user/post/allPosts | Paginated posts |

| | | |
|---|---|---|
| GET | /user/post/userposts/:username | User posts |
| DELETE | /user/post/:id | Delete post |
| POST | /user/post/like/:id | Like/unlike |
| POST | /user/post/comment/:id | Add comment |

Table A3 Saved Posts APIs

| Method | Endpoint | Description |
|---|---|---|
| POST | /user/post/savepost/:id | Save post |
| DELETE | /user/post/savepost/:id | Unsave |
| GET | /user/post/savedposts/:username | Fetch saved |

Table A4 Geo / Connect APIs

| Method | Endpoint | Description |
|---|---|---|
| GET | /user/connect/nearby | Nearby users |
| GET | /user/userlocation | Get location |
| POST | /user/connect/usergeodata | Update location |

Table A5 Community APIs

| Method | Endpoint | Description |
|---|---|---|
| GET | /community | Get communities |
| POST | /community | Create community |
| GET | /community/:id/messages | Fetch chat |

## APPENDIX C: DATABASE SCHEMA (ACTUAL IMPLEMENTATION)

User Model (Key Fields)

- name, username, email
- password (bcrypt hashed)
- avatar, about, coverPhoto
- isVerified, isAdmin
- savedPosts[], posts[]
- location (with 2dsphere indexing)

Post Model

- content (max length validation)
- author (ObjectId ref)
- likes[]
- comments (embedded schema)
- isPublic, isNewsletter

Message Model

- communityId
- senderId
- text
- timestamps

Community Model

- name (unique)
- description
- members[]

# APPENDIX D: CORE SYSTEM WORKFLOW

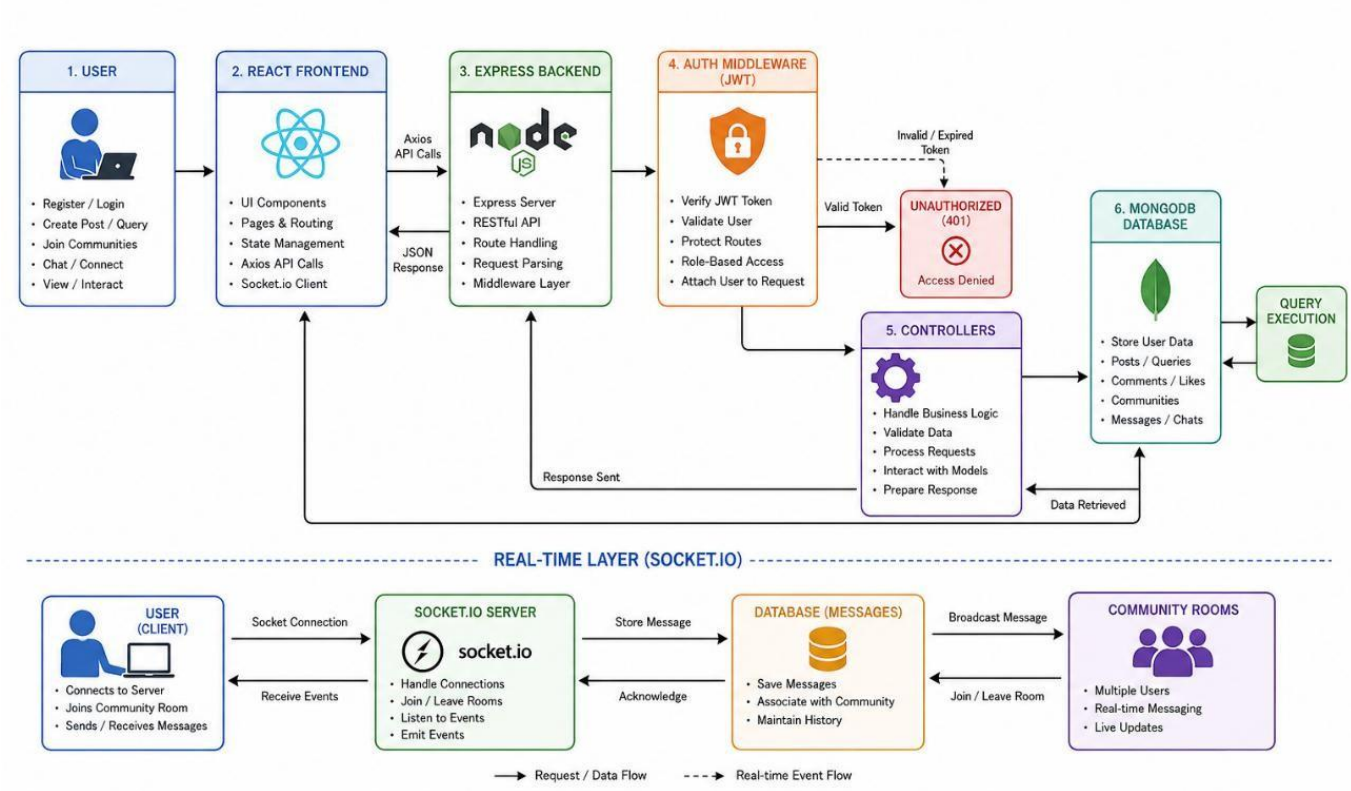


Fig. A1 Core System Workflow of Stackraze Platform

# APPENDIX E: REAL-TIME CHAT FLOW (SOCKET.IO)

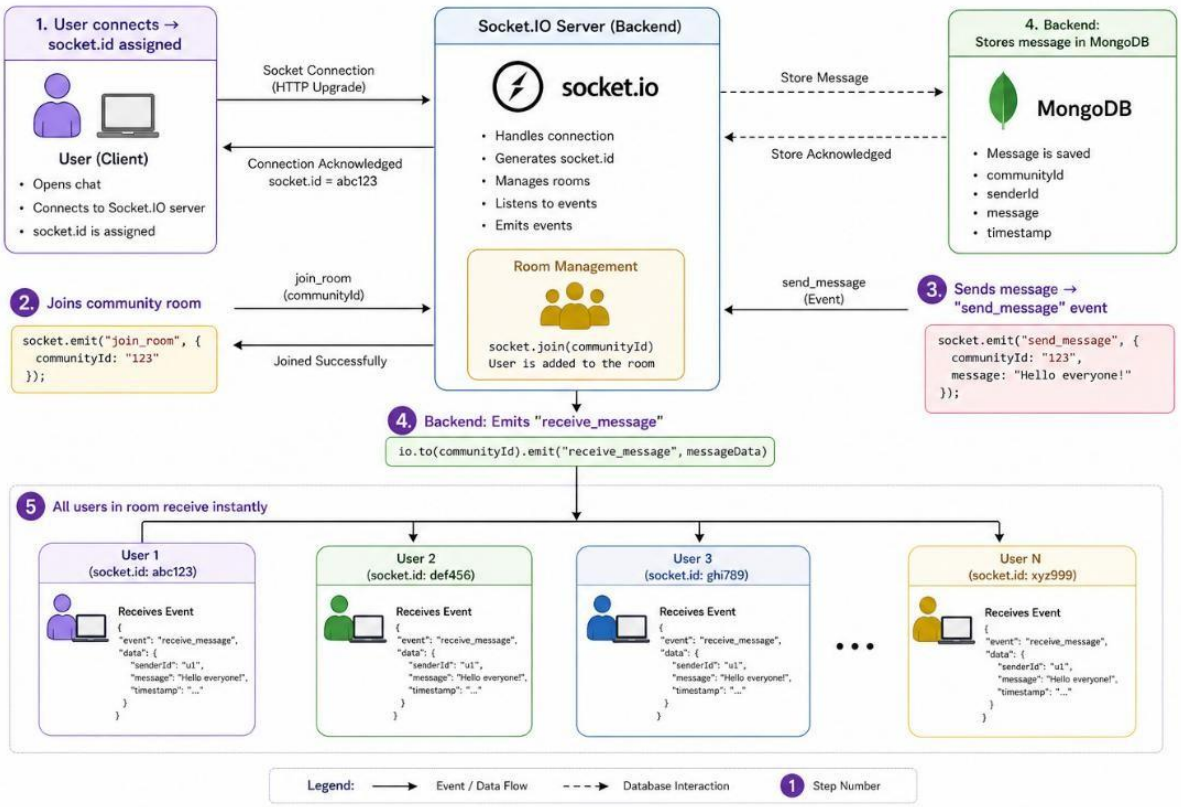


Fig. A2 Real-Time Chat Communication Flow using Socket.io

# APPENDIX F: SECURITY IMPLEMENTATION

- JWT-based authentication (stateless sessions)
- Password hashing using bcrypt
- OTP-based email verification
- Protected routes via middleware
- Input validation & duplicate checks

# APPENDIX G: PROJECT FOLDER STRUCTURE

```
Stackraze/
├── Frontend/
│   └── src/
│       ├── api/              # API call wrappers
│       ├── assets/           # Static assets (images, videos, avatars, logos)
│       ├── components/       # Reusable UI Components
│       │   ├── about/        # About page sections (Hero, People, etc.)
│       │   ├── AuthPages/    # Authentication components
│       │   ├── community/    # CommunityChat, CommunitySidebar
│       │   ├── Connect/      # MapConnect, NearbyUserCard, UpdateLocation
│       │   ├── header/       # Navbar, Profile dropdown
│       │   ├── newsletters/  # NewsCard, SkeletonCard, sample
│       │   ├── post/         # Post cards, comments, upload, etc.
│       │   ├── profile/      # Profile header, tabs, edit modal
│       │   ├── settings/     # Settings options, location, delete account
│       │   ├── ui/           # Specialized UI (theme toggler, map, grids)
│       │   └── [root]        # CheckBoxes, ContactForm, Faqs, Sidebar, etc.
│       ├── config/           # Configurations (api.js)
│       ├── context/          # React Context (AuthContext, PopupContext)
│       ├── hooks/            # Custom hooks (useAvatarUpload)
│       ├── lib/              # Utility libraries
│       ├── pages/            # Route Pages
│       │   ├── about.jsx
│       │   ├── Auth.jsx
│       │   ├── comunity.jsx
│       │   ├── connect.jsx
│       │   ├── contact.jsx
│       │   ├── home.jsx
│       │   ├── Login.jsx
│       │   ├── newsLetters.jsx
│       │   ├── OtpVerification.jsx
│       │   ├── profile.jsx
│       │   ├── settings.jsx
│       │   └── Signup.jsx
│       └── utils/            # Helper functions (avatarHelper, cloudinaryUpload)
│
└── Backend/
    └── src/
        ├── config/           # Environment and passport configs
        ├── controllers/      # Route controllers
        │   ├── admin/
        │   ├── connect/
        │   ├── gridArt/
        │   ├── newsletter/
        │   ├── post/
        │   ├── user/
        │   └── community.js
        ├── mildewares/       # Middlewares (authMiddleware, admin.js)
        ├── models/           # Mongoose Schemas (user, post, community, message...)
        ├── routes/           # Express Routes
        │   ├── admin.js
        │   ├── community.js
        │   ├── news.js
        │   ├── newsletter.js
        │   └── user.js
        ├── utils/            # Backend utilities (emailService)
        ├── views/            # EJS templates (404.ejs)
        ├── app.js            # Main Express app setup
        ├── connectMongo.js   # DB connection logic
        └── sockets.js        # Socket.io configuration
```

Fig. A3 Project Folder Structure